

电机调速实验

北京化工大学

信息科学与技术学院

北京化工大学实践教学中心 请勿盗版

实验任务

在串口要求显示的任务需要输出小组人员姓名学号

任务3-1：用 1 个按钮控制舵机启停。按下按钮时舵机连续可逆旋转，松开按钮时舵机停止旋转。串口监视器中显示角度实时值。

o **任务3-2：**使用 1 个旋转电位器控制舵机旋转到任一角度。串口监视器中显示角度实时值。

o **任务3-3：**根据土壤湿度传感器测得的湿度值确定舵机是否工作。如果舵机工作，每秒钟在 60° 的扇形范围内连续可逆旋转。可以在串口监视器中查看土壤湿度实时值。

o **任务3-4：**用 1 个按钮控制风扇启停。串口监视器中显示风扇状态。

o **任务3-5：**使用 1 个旋转电位器控制风扇转速。串口监视器中显示转速值。

o **任务3-6：**根据温湿度传感器测得的温度值确定风扇是否工作。当温度超过 26°C 时，风扇自动启动；如果温度不超过 30°C ，风扇低速运转；如果温度超过 30°C ，风扇高速运转；如果温度不超过 26°C ，风扇自动停止运转。用 1 个按钮作为停止按钮，当风扇运转时，按下停止按钮，风扇停止运转。

实验任务

任务3-7：智能风扇。

系统上电时为手动模式并且进入停止状态。

(1) 共需要 4 个按钮。1 个按钮负责手动/自动切换，按下按钮系统在手动模式与自动模式之间切换，进入手动模式时进入停止状态，进入自动模式时根据当前条件进行动作；1 个按钮负责风扇高/低速/停止状态切换（手动功能）；1 个按钮负责风扇是否摇头（手动功能）；1 个按钮负责停止所有功能，停止按钮按下停止当前所有动作，再次按下可以使系统继续停止之前的状态。

(2) 手动运行：进入手动运行状态并进入停止状态，使用风扇高速、低速和停止状态切换按钮，可以使风扇高速运转、低速运转或停止转动。使用风扇摇头按钮，可以使舵机连续可逆运行（模拟摇头）或不运行，按下停止按钮可以停止当前所有动作。

(3) 自动运行：根据温湿度传感器测得的温度值确定风扇是否工作。当温度超过 26°C 时，风扇自动启动；如果温度不超过 30°C ，风扇低速运转；如果温度超过 30°C ，风扇高速运转；如果温度不超过 26°C ，风扇自动停止运转。如果风扇正在自动运行，当按下停止按钮时，风扇和舵机停止动作，再次按下停止按钮系统恢复动作。自动运行时默认风扇摇头（舵机连续可逆运行）。

(4) 串口监视器中显示智能风扇的运行状态，如风扇运行还是停止、自动运行还是手动运行、高速运行还是低速运行、是否在摇头、当前室温等。

(5) 程序中需带有小组成员个人身份标识，如姓名首字母缩写、姓氏拼音、名字拼音、学号。

舵机（伺服电机）工作原理及应用

北京化工大学

信息科学与技术学院

北京化工大学实践教学中心 请勿盗版

舵机工作原理及应用

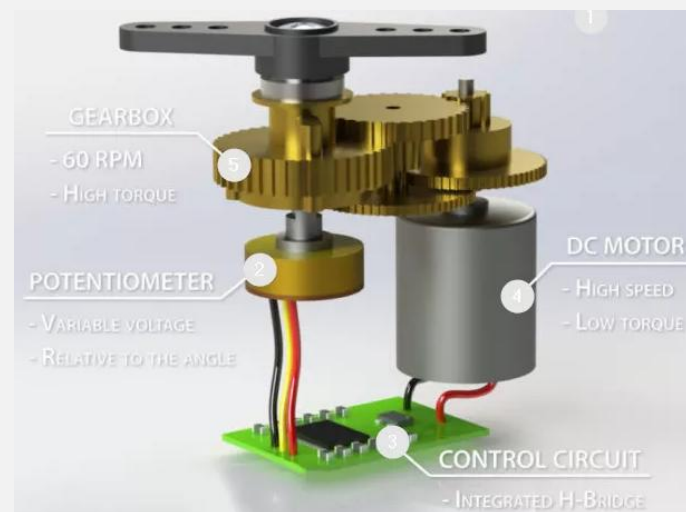
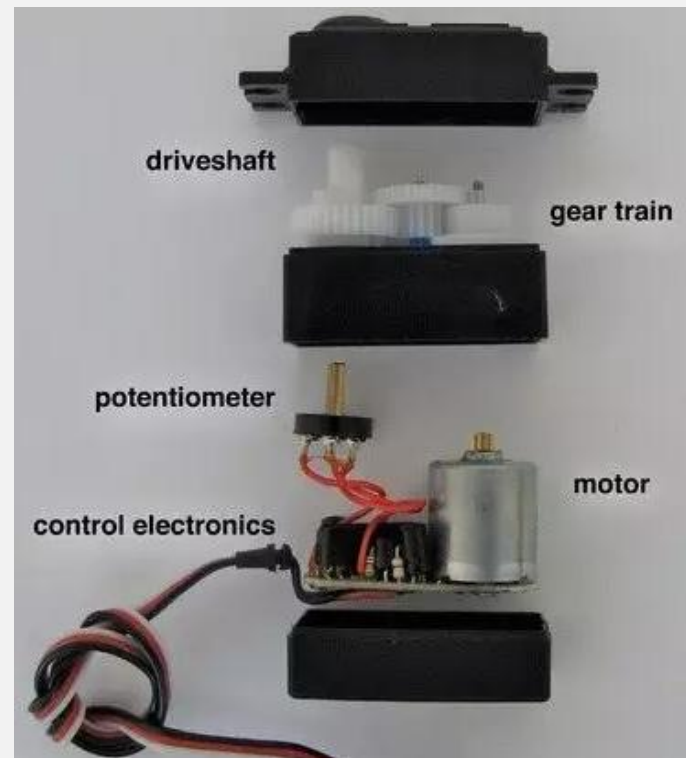
- 1 舵机的结构
- 2 舵机的工作原理
- 3 舵机的分类
- 4 使用Arduino控制舵机的方法
- 5 Arduino官方库Servo
- 6 舵机的接线
- 7 舵机控制示例程序



江工大学 实践教学中心 请勿盗版

1 舵机的结构

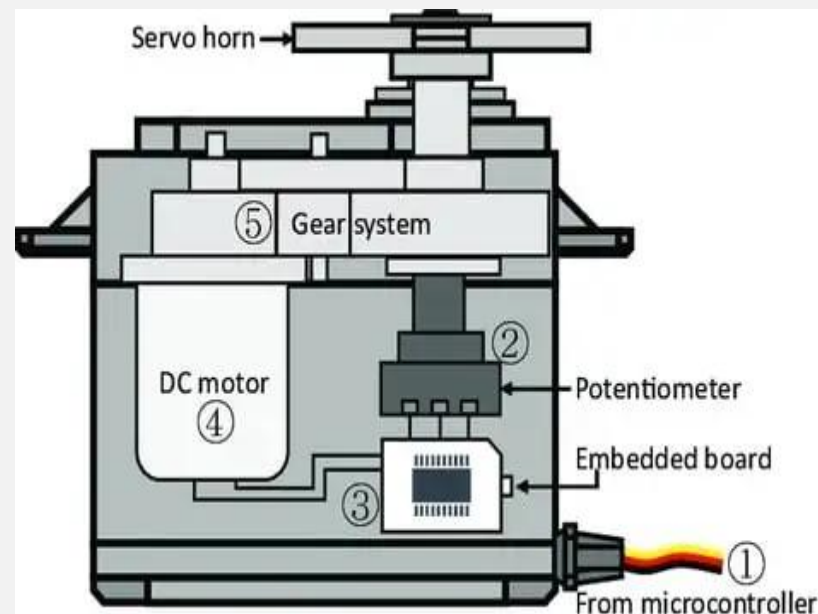
- 伺服电机是带有反馈环节的电机，可以通过伺服电机进行精确的位置控制或者输出较高的扭矩，适用于那些需要角度不断变化并可以保持的控制系统
- 舵机只是一种通俗的叫法，其实质是一个伺服马达，一种低端的伺服电机系统、位置伺服的驱动器，因此其英文叫做 Servo，就是Servo Motor的简称
- 舵机主要由外壳、电路板、无核心马达、齿轮、电位计和嵌入式控制系统构成



1 舵机的结构

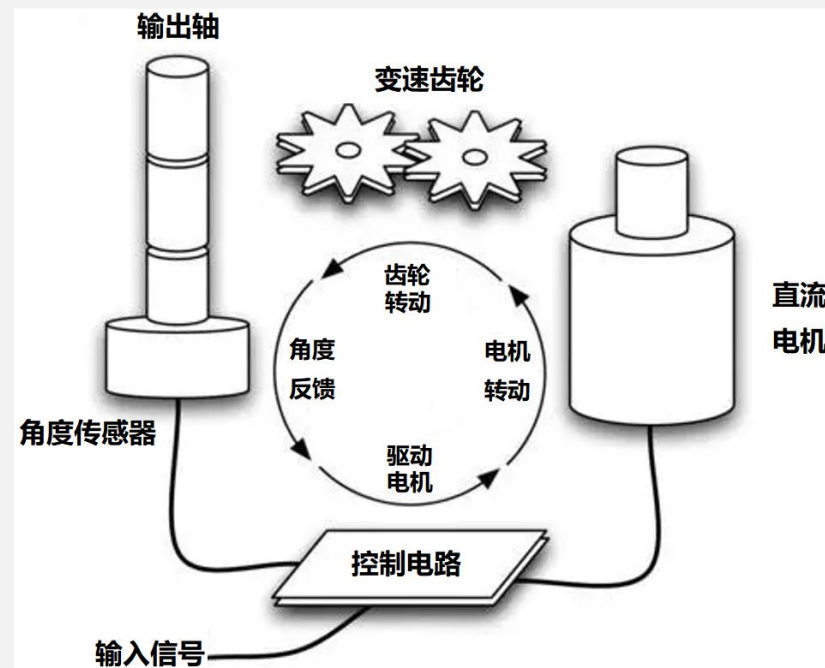
具体结构如图所示：

- 信号线：接收来自（外部）微控制器的控制信号
- 电位器：测量输出轴的位置量，舵机的反馈部分
- 内部控制器：处理来自（外部）微控制器的信号，驱动电机、处理反馈的位置信号，是整个舵机的核心
- 电机：作为执行机构，输出转速、转矩、位置
- 传动机构：根据一定传动比，将电机输出的行程缩放到最终输出的角度上



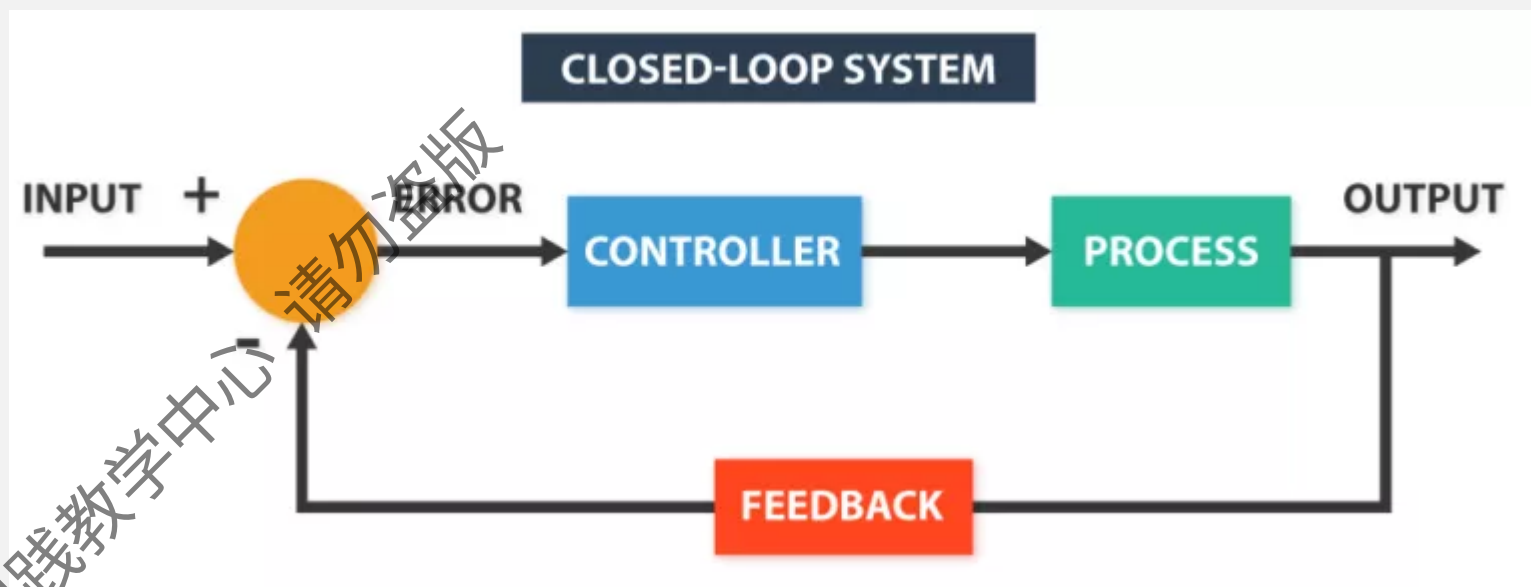
1 舵机的结构

- o 变速箱降低了电机的转速，从而增加了输出轴上的输出扭矩。输出轴的最大速度通常约为60RPM
- o 舵机旋转带动内部可调电位器（旋转电阻）变化，根据变化的电阻可以确定目前舵机的角度
- o 舵机将PWM信号与滑动变阻器的电压相比对，通过硬件电路实现固定控制增益的位置控制。也就是说，它包含了电机、传感器和控制器，是一个完整的伺服电机（系统）



1 舵机的结构

- 电机输出轴的位置由内部电位计不断采样测量，并与微控制器（例如STM32、Arduino）设置的目标位置进行比较。根据相应的偏差，控制设备会调整电机输出轴的实际位置，使其与目标位置匹配，这样就形成了闭环控制系统

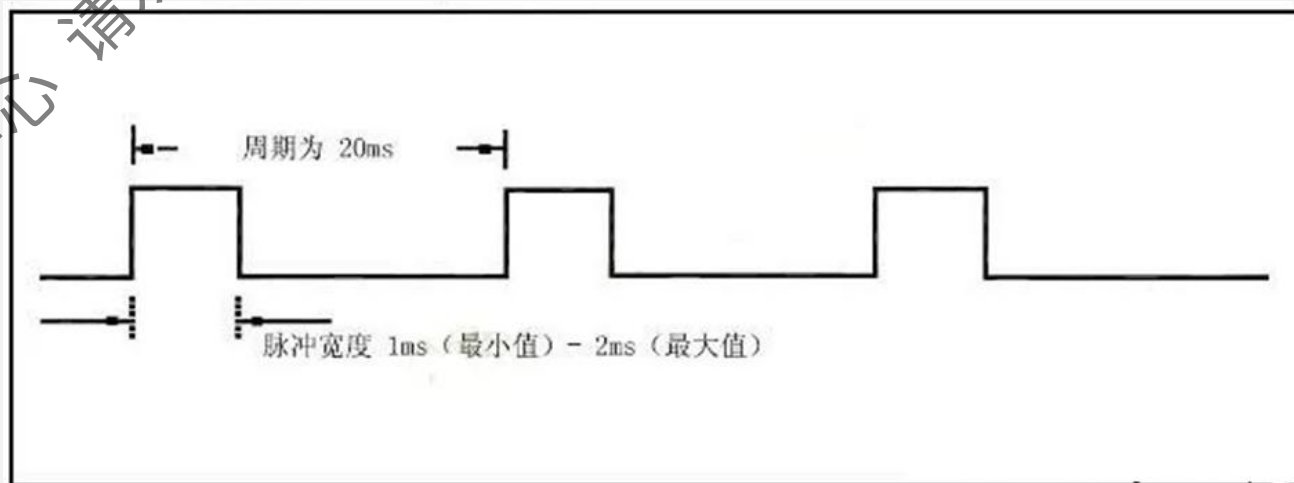


2 舵机的工作原理

- ① 在舵机内部有一个基准电压，微处理器产生的PWM信号通过信号线进入舵机、产生直流偏置电压，与舵机内部的电位器的电压比较，获得电压差输出
- ② 电压差的正负输出到电机驱动芯片上，从而决定正反转
- ③ 再驱动无核心马达开始转动，透过减速齿轮将动力传至摆臂，同时由电位器（位置检测器）送回信号，判断是否已经到位
- ④ 当舵机开始旋转的时候，舵机内部通过级联减速齿轮带动电位器旋转，使得电压差为零时，电机停止转动

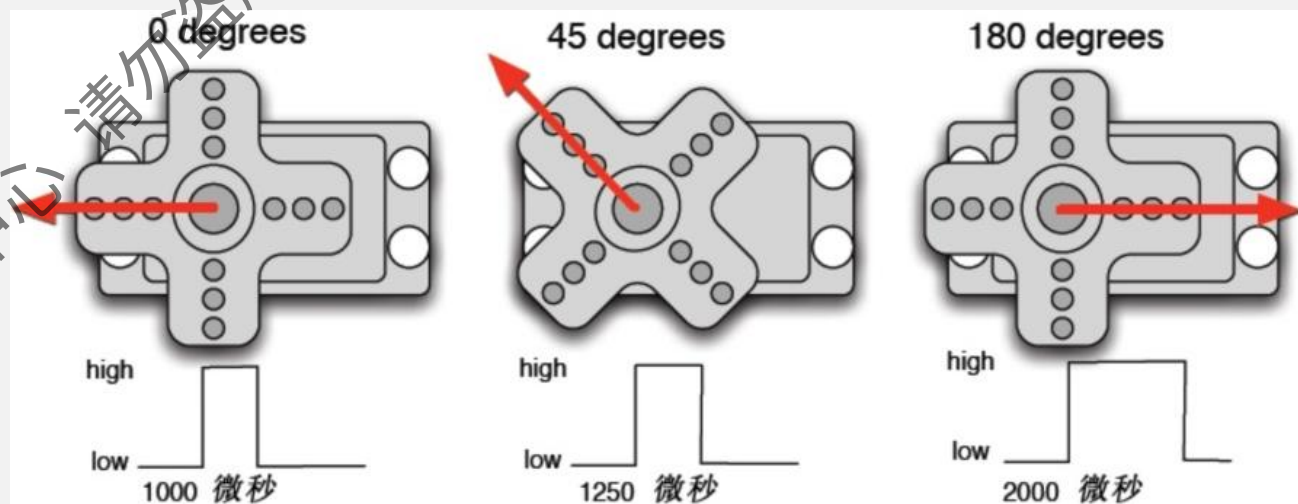
2 舵机的工作原理

- 通过向舵机的信号线（控制线）发送PWM信号来控制舵机的输出量，这种控制方法叫做脉冲宽度调制（PWM）。PWM的周期以及占空比是可控的，通过调节PWM信号的占空比、即脉冲的长短，决定舵机转动多大角度、决定舵机输出轴的位置
- 通常舵机的基准信号都是周期为20ms（50Hz）、宽度为1.5ms，这个基准信号定义的位置为中间位置
- 舵机有最大转动角度，中间位置的定义就是从这个位置到最大角度与最小角度的量完全一样
- 最重要的一点，不同舵机的最大转动角度可能不同，但其中间位置的脉冲宽度是一定的，那就是1.5ms



2 舵机的工作原理

- 不同品牌，甚至同一品牌的不同舵机，都会有不同的最大转动角度和最小转动角度，也会有不同的最大脉冲和最小脉冲。一般而言，最小脉冲为1ms，最大脉冲为2ms
- 当控制系统发出指令，让舵机移动到某一位置并保持这个角度，这时外力的影响不会让角度产生变化，但是这个是有上限的，上限就是舵机的最大扭力。除非控制系统不停地发出脉冲稳定舵机的角度，舵机的角度不会一直不变



2 舵机的工作原理

- 向舵机发送脉冲宽度为1ms的信号时，舵机的输出轴将移至最小的位置（0°）



- 向舵机发送脉冲宽度为1.5ms的信号时，舵机的输出轴将移至中间位置（90°）

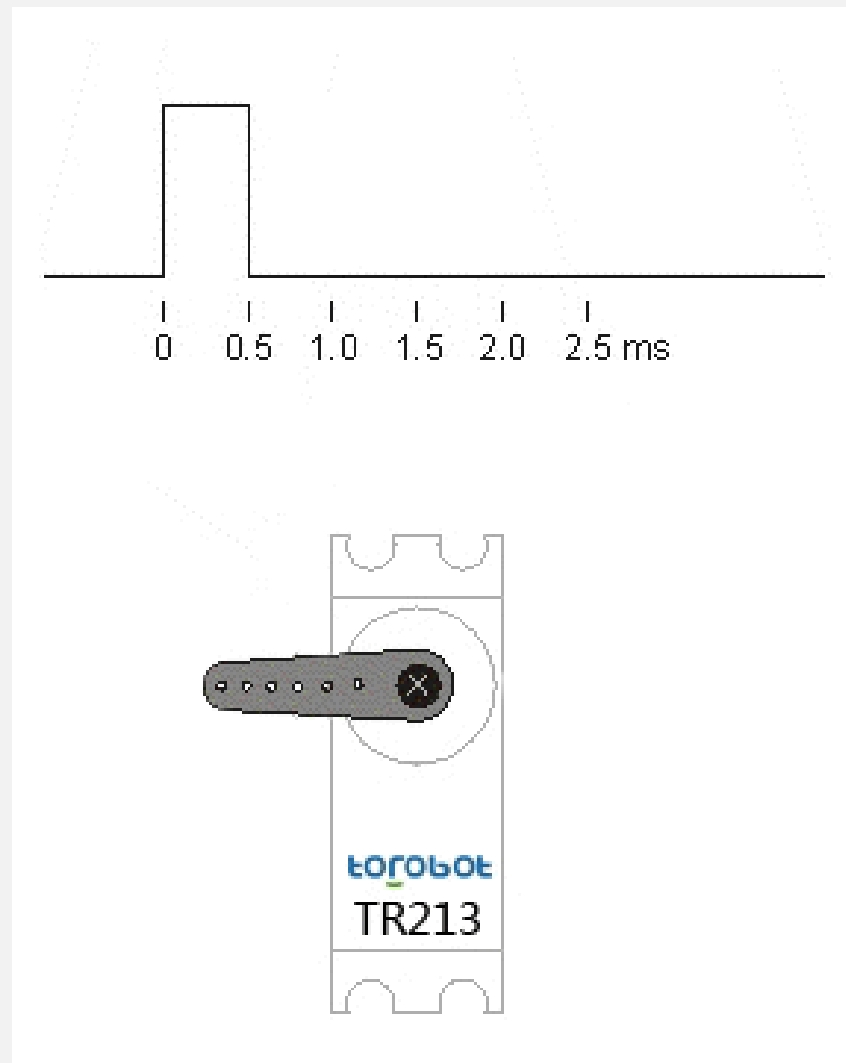


- 向舵机发送脉冲宽度为2ms的信号时，舵机的输出轴将移至最大的位置（180°）



2 舵机的工作原理

- 理论上脉宽分布应该在1~2ms之间，实际上可由0.5ms~2.5ms之间、脉宽与舵机的转角 0° ~ 180° 相对应
- 当舵机接收到一个小于1.5ms的脉冲，输出轴会以中间位置为标准，逆时针旋转一定角度。接收到的脉冲大于1.5ms情况相反
- 以 180° 舵机为例，脉冲宽度与舵机输出轴的位置之间的关系如下：
 - 0.5ms —— 0°
 - 1ms —— 45°
 - 1.5ms —— 90°
 - 2ms —— 135°
 - 2.5ms —— 180°



3 舵机的分类

- 舵机分为模拟舵机和数字舵机

- 模拟舵机：需要不断地发送目的地PWM信号，才能旋转到指定位置

- 例如：需要舵机旋转 90° ，就要不断地发送 90° 的PWM信号直到到达指定位置才能停止

- 数字舵机：只需给一个目的地PWM信号，即可旋转到指定位置

- 例如：需要舵机旋转 90° ，只需发送一次 90° 的PWM信号，它就可以旋转到 90°

3 舵机的分类

- SG90是在Arduino设计中比较常用的一款小舵机，价格低廉、结构紧凑，但精度很低、位置镇定能力较差，适合一些载重比较轻的场合
- 绝大多数舵机只能在 $0^{\circ}\sim 180^{\circ}$ 之间旋转，SG90的转动角度只有 180°
- SG90是一种模拟舵机，所以在使用时需要不停地向舵机发送位置脉冲信号

Operating voltage	4.8 V
Weight	9 g
Stall torque	1.8 kg/cm (4.8 V)
Gear type	POM gear set
Operating speed	0.12 sec/ 60° (4.8 V)
Operating temperature	0 – 55 $^{\circ}\text{C}$

3 舵机的分类

- 舵机控制板输出的是PWM信号，所以舵机控制板都可以控制180°和360°舵机
- 360°舵机与一般舵机的区别是：
 - 给180°舵机一个PWM信号，舵机会转到一个特定角度。PWM信号控制它的旋转角度，0.5ms~2.5ms的PWM对应控制180°舵机的0~180°，是一一对应的，一个PWM值对应舵机的一个角度
 - 360°舵机就是一个普通直流电机和驱动板的组合，所以它只能连续旋转，不能定位
 - PWM信号控制360°舵机的旋转速度和旋转方向，0.5ms~1.5ms的PWM控制它正转，值越小，旋转速度越大；1.5ms~2.5ms的PWM控制它反转，值越大，旋转速度越大；1.5ms的PWM控制它停止

4 使用Arduino控制舵机的方法

用Arduino控制舵机一般有两种方法：

- ① 通过Arduino的普通数字引脚产生占空比不同的方波，模拟产生PWM信号进行舵机控制
- ② 直接利用Arduino自带的Servo库进行控制

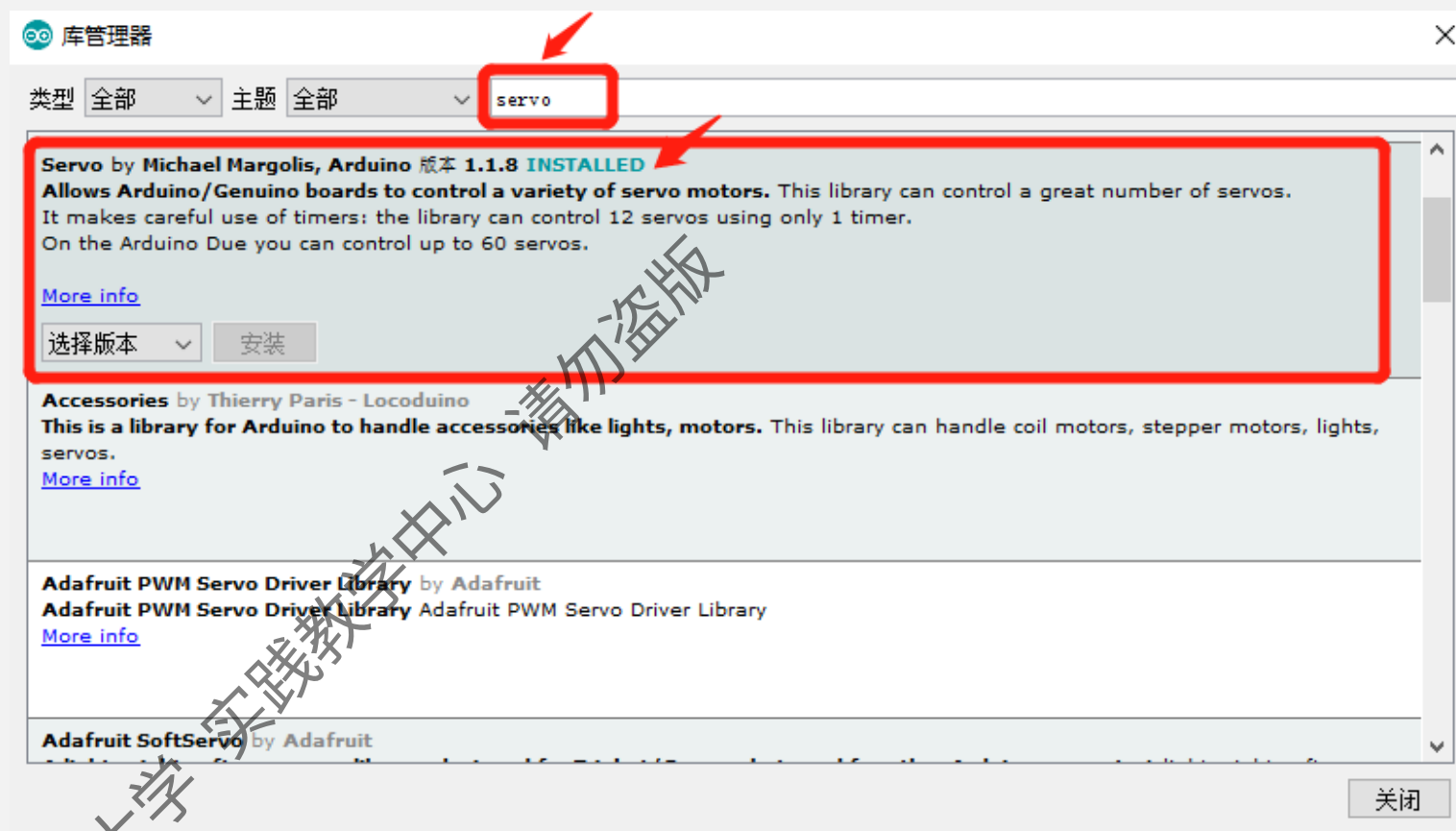
优点：程序编写方便

缺点：只能控制2路舵机，因为Arduino自带函数只能利用数字9、10接口

Arduino的驱动能力有限，当需要控制1个以上的舵机时，需要外接电源

5 Arduino官方库Servo

- 打开Arduino软件，点击工具→管理库，打开库管理器，搜索servo，点击下载，出现INSTALLED即为下载成功



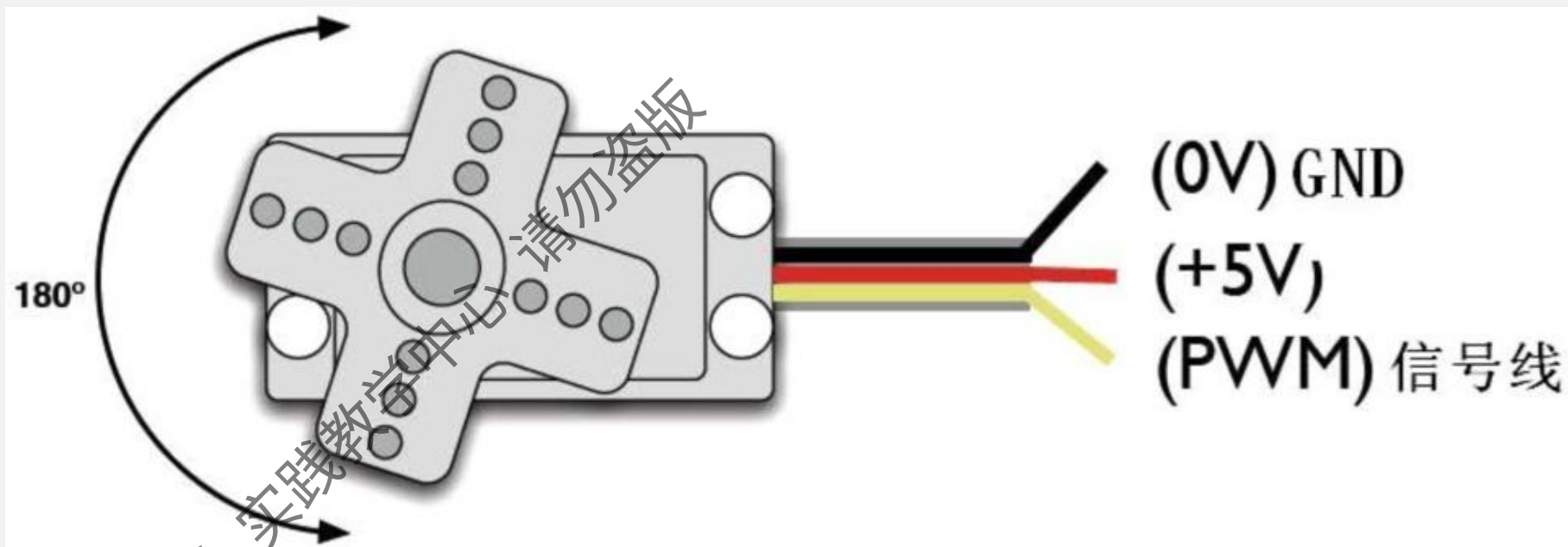
5 Arduino官方库Servo

o Servo库常用函数：

- o attach()：设定舵机接口，连接舵机，只能选择数字引脚9或者10
- o write()：角度控制，用于设定舵机旋转的角度，可设定范围0°~180°
- o read()：用于读取上一次舵机转动角度，可理解为读取最后一条write()命令中的值
- o writeMicroseconds()：利用微秒值控制角度
- o detach()：断开舵机连接

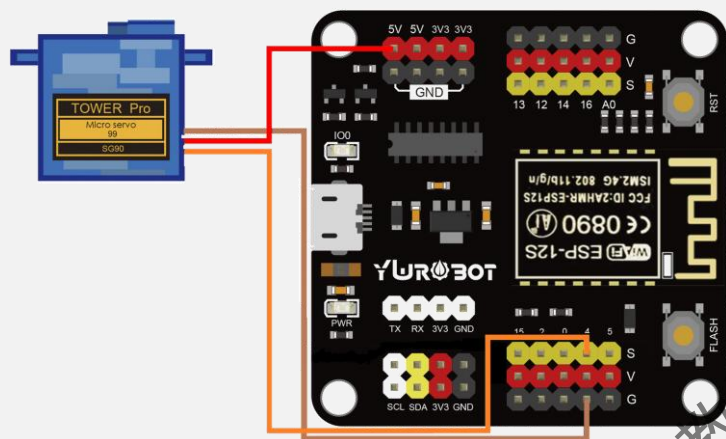
6 舵机的接线

- 舵机一般都外接三根线，分别用棕（黑）、红、橙（黄）三种颜色进行区分，由于品牌不同，颜色也会有所差异，棕（黑）色为接地线，红色为电源正极线，橙（黄）色为信号线
- 由于Arduino的驱动能力有限，当需要控制多个舵机时，需要外接电源以及使用专用的舵机控制板

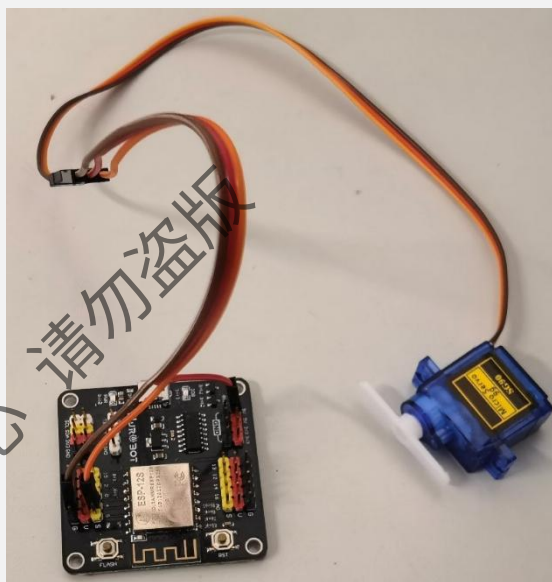


6 舵机的接线

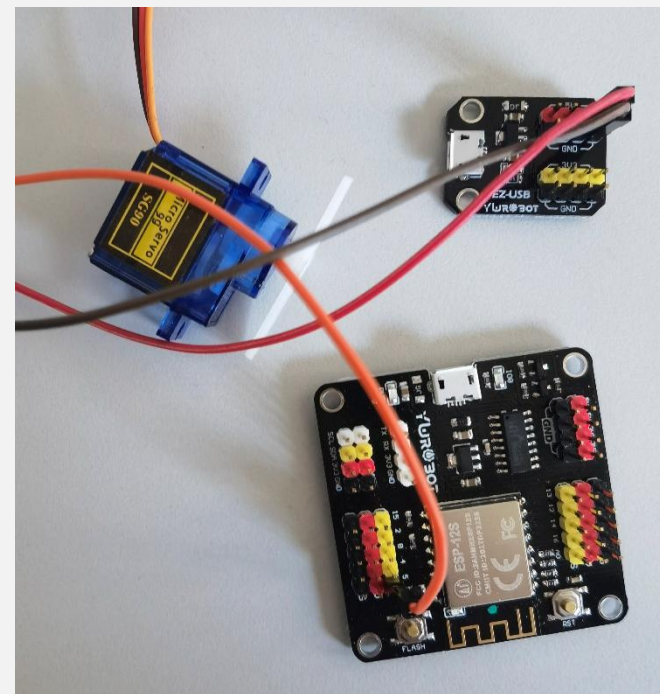
- 采用YwRobot ESP8266开发板时，舵机红色线接开发板5V，棕色线接开发板GND，橙色信号线接开发板数字引脚（15除外）



实验原理图



实物连接图1



实物连接图2

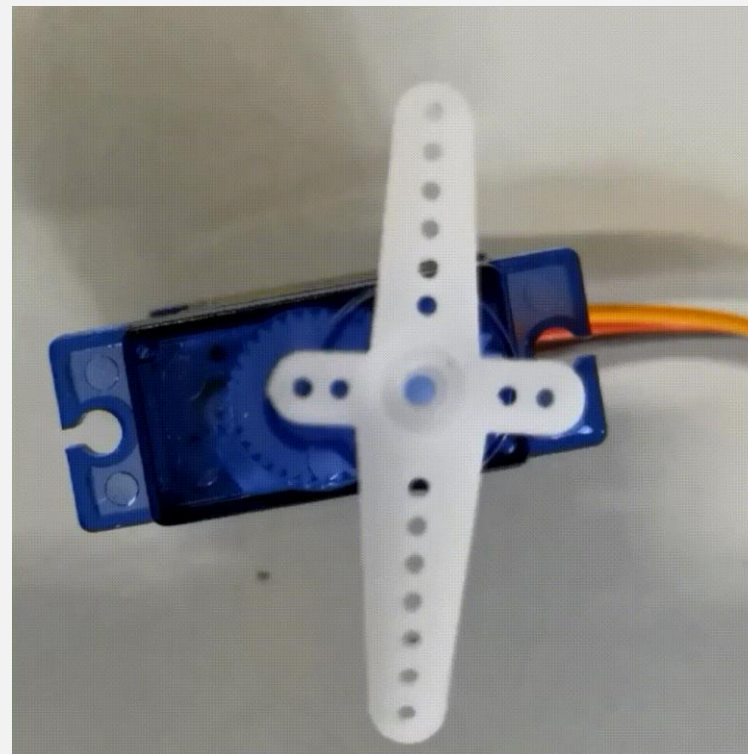
7 舵机控制示例

1. 舵机连续可逆旋转
2. 使用按钮控制舵机
3. 使用旋转电位器控制舵机
4. 根据土壤湿度控制舵机



7.1 舵机连续可逆旋转

```
1. #include <Servo.h>
2. Servo myservo; //创建一个舵机控制对象
3. int pos = 0; // 该变量用于存储舵机角度位置
4. /*~~~~~分割线~~~~~ */
5. void setup()
6. {
7.   myservo.attach(4); //舵机控制线连接开发板数字4脚
8. }
9. /*~~~~~分割线~~~~~ */
10. void loop()
11. {
12.   for(pos = 0; pos < 180; pos += 1) // 从0°到180°运动
13.   {
14.     myservo.write(pos); // 指定舵机转向的角°, 写入角度值
15.     delay(15); // 等待15ms让舵机到达指定位置
16.   }
17.   for(pos = 180; pos >= 1; pos -= 1) //从180°到0°运动
18.   {
19.     myservo.write(pos); // 指定舵机转向的角度, 写入角度值
20.     delay(15); // 等待15ms让舵机到达指定位置
21.   }
22. }
```



实验现象：

舵机在0~180°间连续可逆转动

7.2 使用按钮控制舵机

```
1. #include <Servo.h>
2. #define BUTTON 0 //令变量BUTTON为0, 0对应ESP8266开发板上端口0
3. Servo myservo;    //创建一个舵机控制对象
4. int pos;           //用于存储舵机角度位置的变量
5. void setup(){
6.   pinMode(BUTTON,INPUT); //定义按钮为输入
7.   myservo.attach(5);      //舵机控制线连接开发板数字5脚
8.   Serial.begin(9600);     //串口波特率设置为9600
9.   void loop(){
10.    if (digitalRead(BUTTON) == 0){ //按下按钮时, 舵机连续可逆旋转
11.      for(pos=0;pos<180;pos++){ //从0°到180°运动, 每次步进1°
12.        myservo.write(pos);    //向舵机写入角度值
13.        delay(20);             //等待20ms让舵机到达指定位置
14.        Serial.print("舵机当前角度值: "); //串口输出舵机当前角度值
15.        Serial.println(pos);
16.      } for(pos=179;pos>=0;pos--){ //从180°到0°运动, 每次步进1°
17.        myservo.write(pos);
18.        delay(20);
19.        Serial.print("舵机当前角度值: ");
20.        Serial.println(pos);
21.      } else{
22.        myservo.write(0);
23.        delay(3000); //舵机静止时, 每隔3s向串口输出提示信息
24.        Serial.println("按钮未按下, 舵机静止、当前角度值: 0");}}
```

接线方式

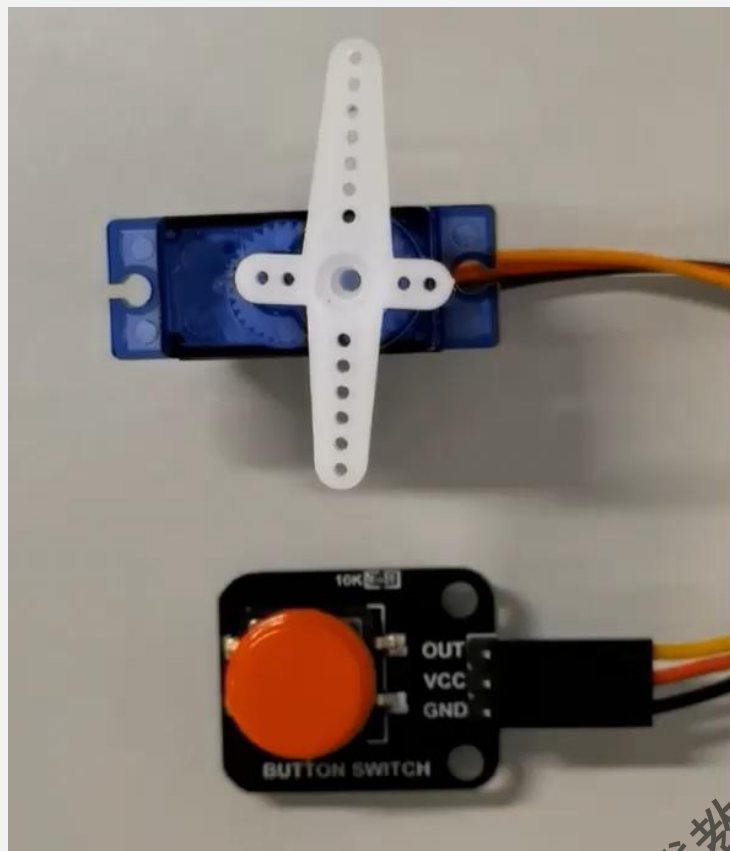
舵机

- 红色线接开发板5V
- 棕色线接开发板GND
- 橙色信号线接开发板信号引脚5

按钮

- OUT引脚接开发板信号引脚0
- VCC引脚接开发板3.3V
- GND引脚接开发板GND

7.2 使用按钮控制舵机



实验现象：按下按钮时，舵机在0~180°间连续可逆转动；松开按钮时，舵机转回到0°后停止

7.3 使用旋转电位器控制舵机

```
1. #include <Servo.h>
2. Servo myservo; //创建一个舵机控制对象
3. int pos;        //用于存储舵机角度位置的变量
4. void setup()
5. {
6.   myservo.attach(4); //舵机控制线连接开发板数字4脚
7.   Serial.begin(115200); //串口波特率设置为115200
8.   pinMode(A0,INPUT); //定义A0模拟口为输入
9. }
10. void loop()
11. {
12.   pos=map(analogRead(A0), 0, 1023, 0, 180); //读取模拟口A0的数值, 并将其从
        0~1023的范围变换到0~180的范围
13.   myservo.write(pos); //向舵机写入角度值
14.   Serial.print("舵机当前角度值: "); //串口输出舵机当前角度值
15.   Serial.println(pos);
16.   delay(200); //实验时, 建议将200ms依次改为20ms、2000ms, 观察旋转电位器旋钮
        时舵机的响应情况
17. }
```

接线方式

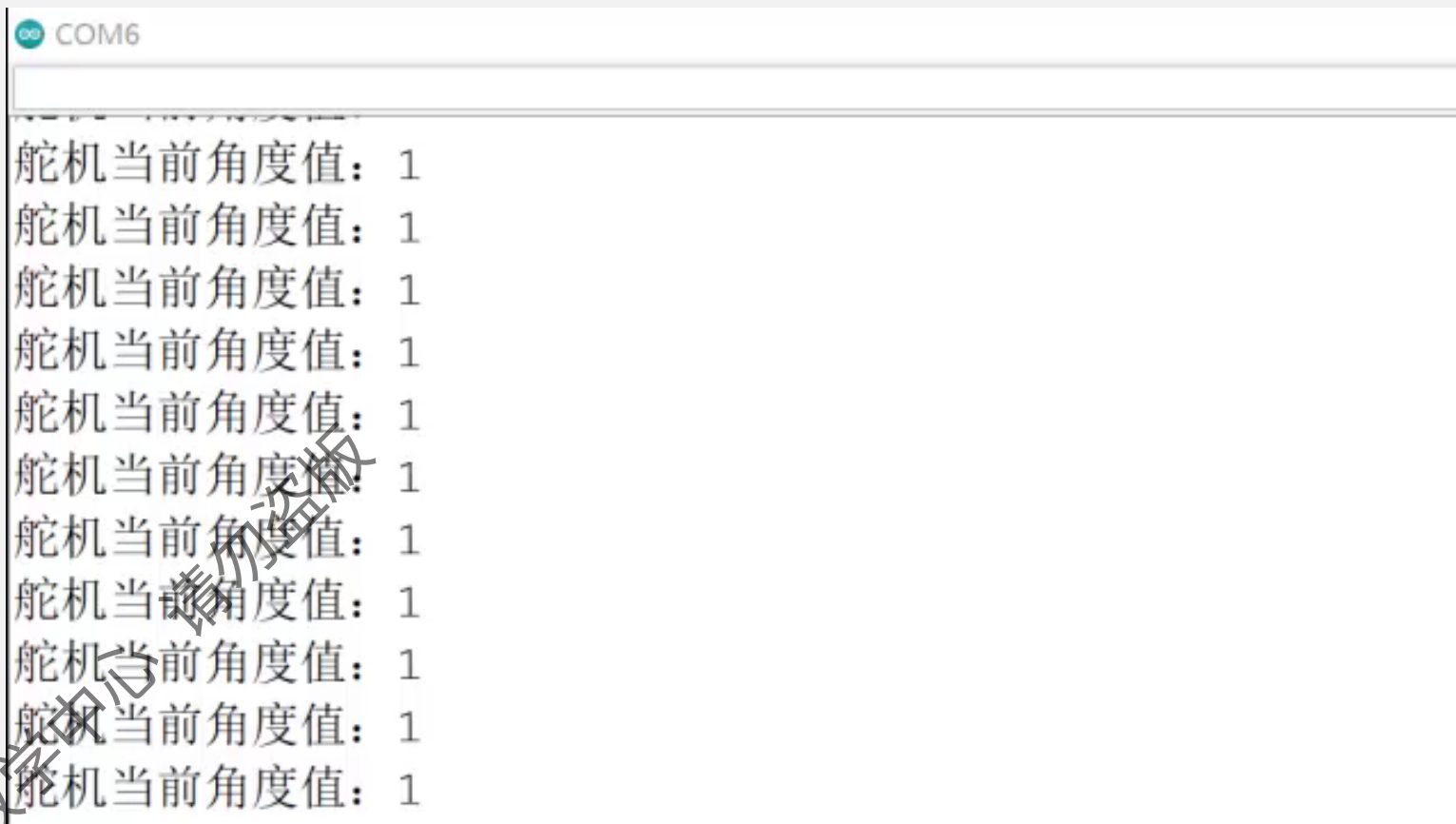
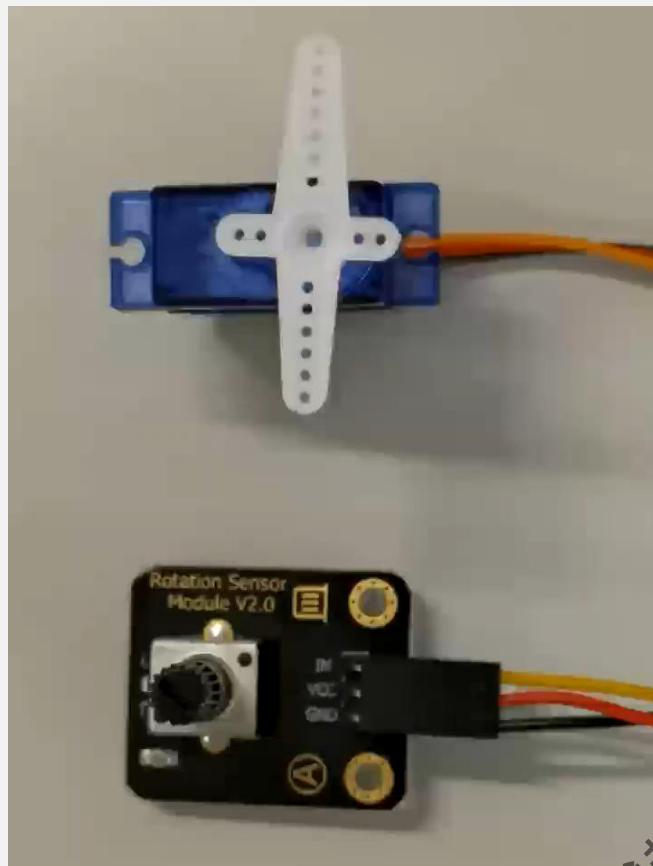
舵机

- 红色线接开发板5V
- 棕色线接开发板GND
- 橙色信号线接开发板信号引脚4

旋转电位器

- IN引脚接开发板信号引脚A0
- VCC引脚接开发板3.3V
- GND引脚接开发板GND

7.3 使用旋转电位器控制舵机



实验现象：舵机角度随旋转电位计数值在0~180°间递增或递减

7.4 根据土壤湿度控制舵机

```
1. #include <Servo.h>
2. Servo myservo; //创建一个舵机控制对象
3. void setup(){
4.   myservo.attach(4); //舵机控制线连接开发板数字4脚
5.   Serial.begin(19200); //串口波特率设置为19200
6.   pinMode(A0,INPUT); //定义A0模拟口为输入
7. }
8. void loop(){
9.   if (analogRead(A0) < 15) //土壤湿度临界值根据实际情况自行调整
10.  {
11.    myservo.write(0); //向舵机写入角度值0°
12.    delay(500);
13.    myservo.write(60); //0.5s后让舵机到达60°
14.    delay(500);
15.  } else{
16.    myservo.write(0);
17.    delay(3000);
18.    Serial.print("当前土壤湿度值: "); //串口输出当前土壤湿度值
19.    Serial.println(analogRead(A0));}
```

实验现象：土壤湿度低于15时，舵机每秒在60°扇形区域内可逆旋转；
土壤湿度超过15时，舵机回到0°停止工作

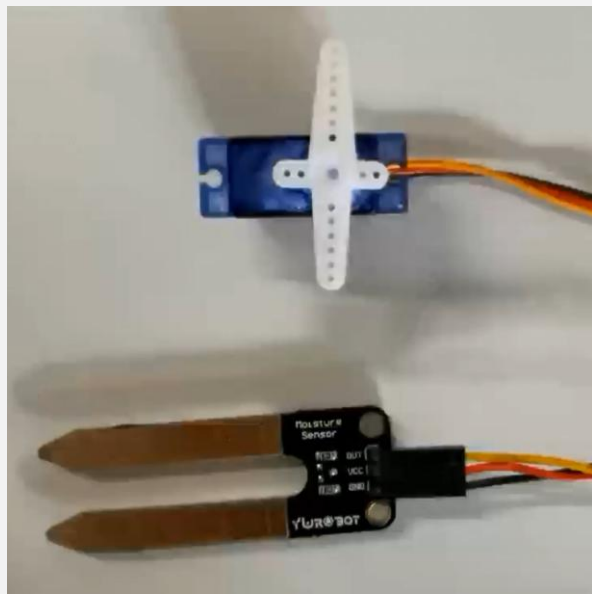
接线方式

舵机

- 红色线接开发板5V
- 棕色线接开发板GND
- 橙色信号线接开发板信号引脚4

土壤湿度传感器

- OUT引脚接开发板信号引脚A0
- VCC引脚接开发板3.3V
- GND引脚接开发板GND



风扇（直流减速电机）

工作原理及应用

北京化工大学

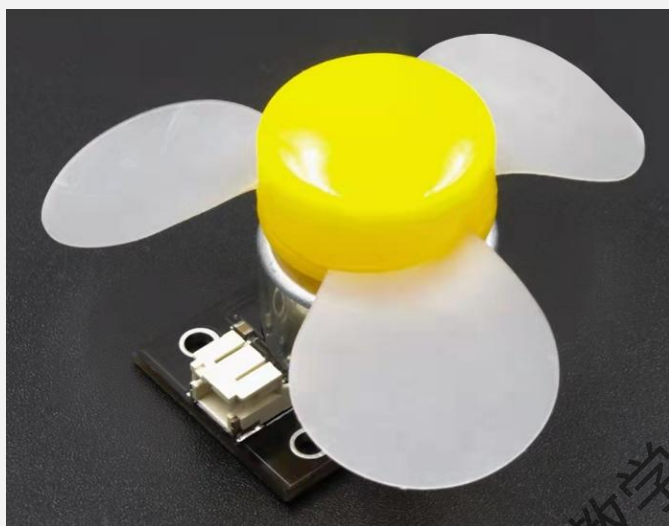
信息科学与技术学院

化工大学实践教学中心

请勿盗版

风扇（直流减速电机）工作原理及应用

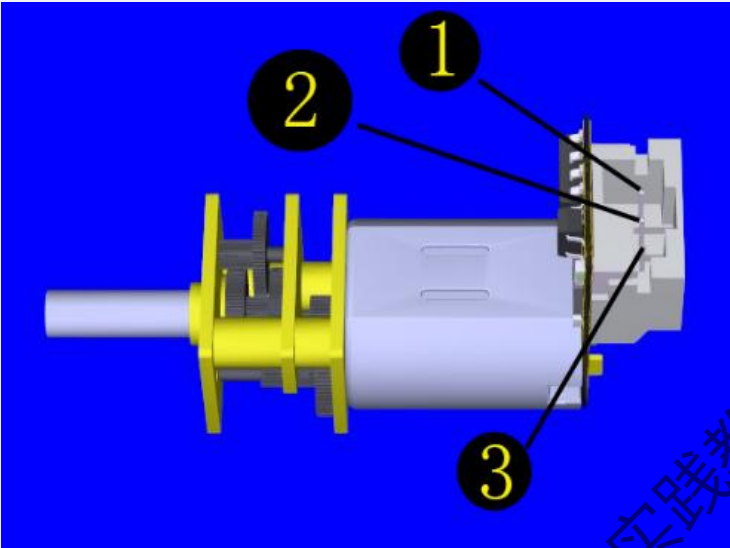
- 01 风扇的结构
- 02 直流减速电机的工作原理
- 03 风扇控制示例程序



1 风扇的结构

- YwRobot风扇模块采用R300电机和软硅胶风扇构成
- 不带驱动板，信号采用数字输入
- 正反转采用手动控制

名称 / EZ MOTOR R300 风扇模块
尺寸 / 33*22mm
固定孔 / 3mm
孔距 / 700mil 17.78mm
风扇 / 软硅胶材质
内置 / R300 马达
电压 / 5V
电流 / <300mA (5V)
平台 / Arduino、单片机

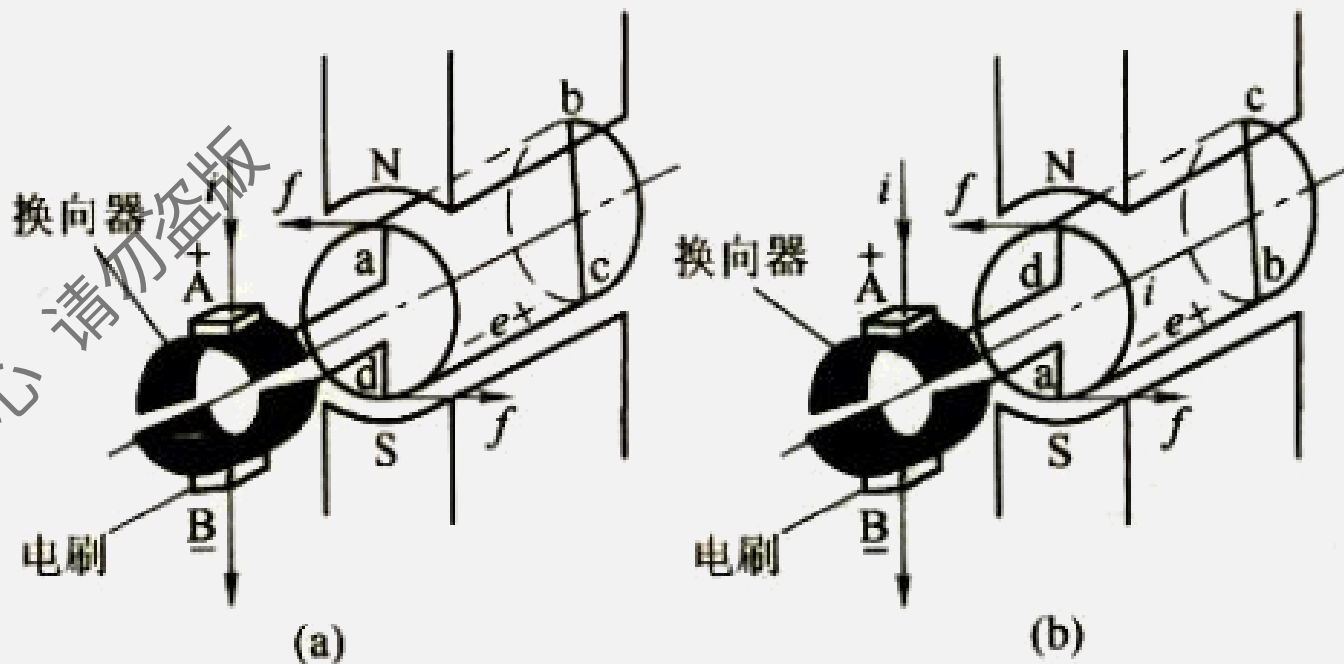
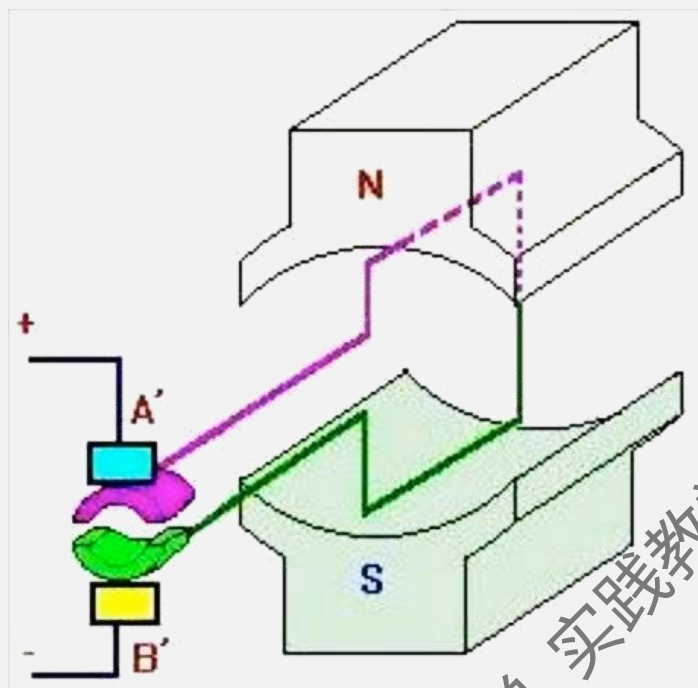


标号	名称	功能描述
1	信号	PWM信号输入
2	电源	VCC
3	地	GND



2 直流减速电机的工作原理

- 转子（电枢）线圈通入直流电流，导体在定子（励磁）磁场中受电磁力作用，产生电磁转矩，线圈在此转矩作用下旋转
- 虽然导体中流通的电流为交变的，但是电磁转矩方向不变



3 风扇控制示例

1. 使用按钮控制风扇启停
2. 使用旋转电位器控制风扇转速
3. 根据室温控制风扇

风扇采用3P防反接杜邦线与
YwRobot ESP8266开发板连接，
信号线接开发板数字引脚

3.1 使用按钮控制风扇启停

```
1. #define FANPIN 16 //定义风扇信号线接口
2. #define BUTTON 0 //定义按钮信号线接口
3.
4. void setup()
5. {
6.   pinMode(BUTTON,INPUT); //定义按钮为输入
7.   Serial.begin(9600); //串口波特率设置为9600
8. }
9.
10. void loop()
11. {
12.   if (digitalRead(BUTTON) == 0) //按下按钮时，风扇旋转
13.   {
14.     analogWrite(FANPIN,1000); //将转速值1000写入风扇信号口
15.     delay(2000);
16.     Serial.println("风扇正在工作");
17.   }
18.   else
19.   {
20.     analogWrite(FANPIN,0); //将转速值0写入风扇信号口
21.     delay(2000);
22.     Serial.println("风扇未转动");
23.   }
24. }
```

接线方式

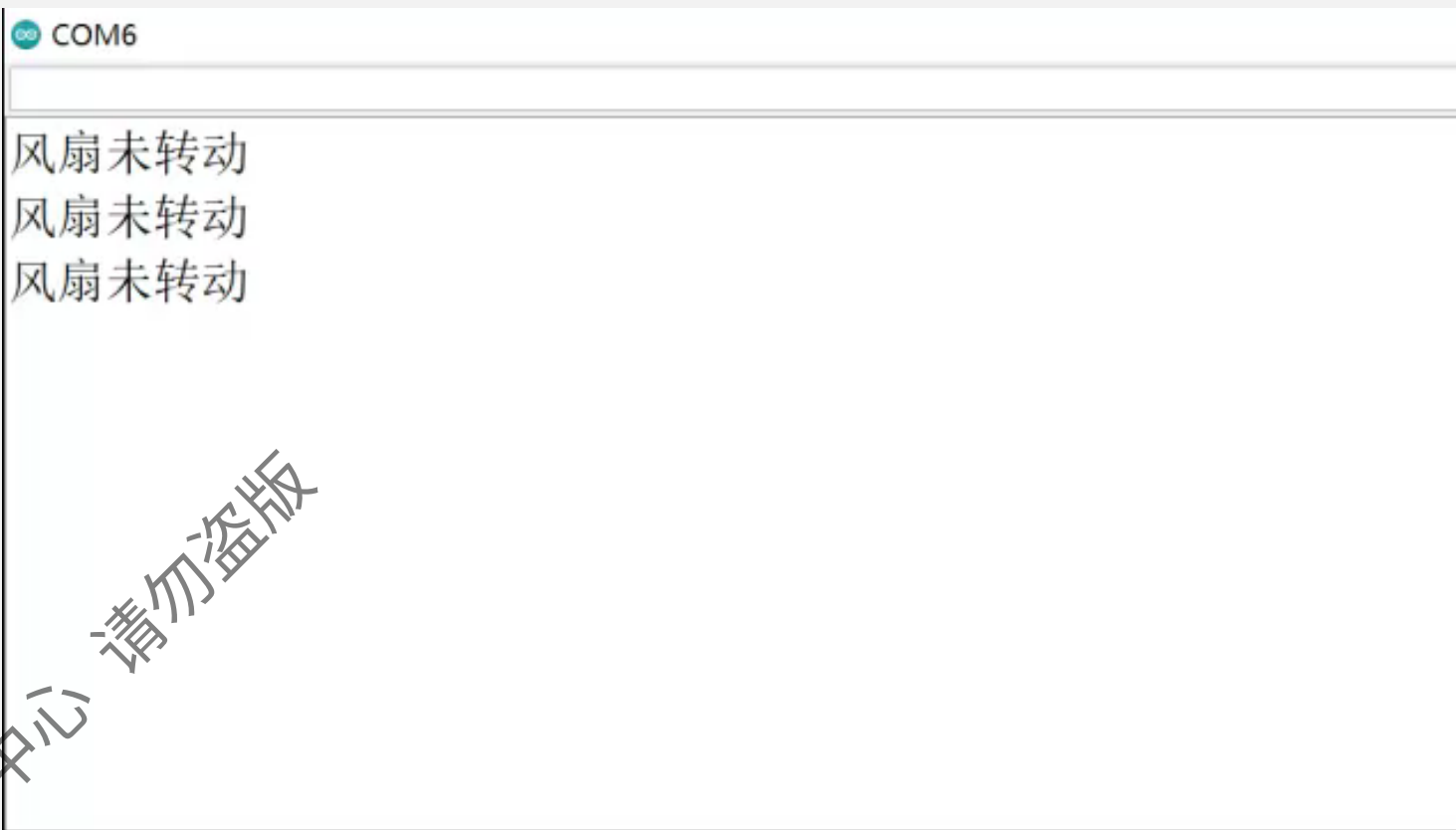
风扇

- 橙色信号线接开发板信号引脚16
- 红色线接开发板3.3V
- 棕色线接开发板GND

按钮

- OUT引脚接开发板信号引脚0
- VCC引脚接开发板3.3V
- GND引脚接开发板GND

3.1 使用按钮控制风扇启停



实验现象：按下按钮时，风扇连续恒速单向转动；松开按钮时，风扇停止转动

3.2 使用旋转电位器控制风扇转速

```
1. #define FANPIN 16 //定义风扇信号线接口
2. int speed; //用于存储风扇转速值
3.
4. void setup()
5. {
6.   Serial.begin(115200); //串口波特率设置为115200
7.   pinMode(A0,INPUT); //定义A0模拟口为输入
8. }
9.
10. void loop()
11. {
12.   speed=map(analogRead(A0), 0, 1023, 0, 1500); //读取模拟口A0的数值, 并将其从
        0~1023的范围变换到0~1500的范围
13.   analogWrite(FANPIN,speed); //向风扇写入速度值
14.   Serial.print("风扇当前速度: "); //串口输出风扇当前速度值
15.   Serial.print(speed);
16.   Serial.println("r/min");
17.   delay(200);
18. }
```

接线方式

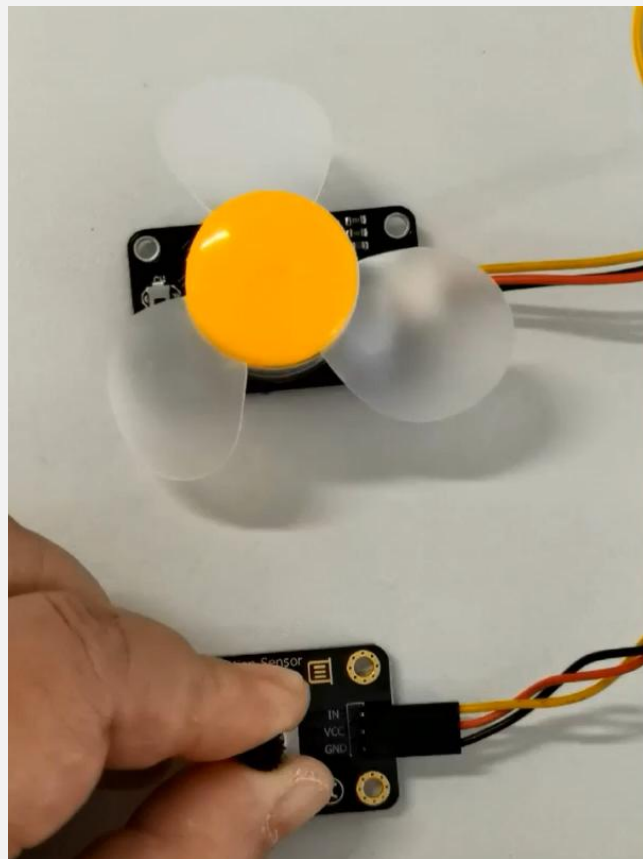
风扇

- 橙色信号线接开发板信号引脚16
- 红色线接开发板3.3V
- 棕色线接开发板GND

旋转电位器

- IN引脚接开发板信号引脚A0
- VCC引脚接开发板3.3V
- GND引脚接开发板GND

3.2 使用旋转电位器控制风扇转速



COM6

风扇当前速度: 4r/min
风扇当前速度: 6r/min
风扇当前速度: 6r/min
风扇当前速度: 6r/min
风扇当前速度: 6r/min
风扇当前速度: 6r/min
风扇当前速度: 6r/min
风扇当前速度: 6r/min
风扇当前速度: 6r/min
风扇当前速度: 6r/min

实验现象：风扇转速随旋转电位计数值的增加而增加、减少而减少

3.3 根据室温控制风扇

DHT11_fan §

```
#include <DHT.h>
#define BUTTON 0 //定义按钮信号线接口
#define FANPIN 14 //定义风扇信号线接口
#define DHTPIN 2 //定义温湿度传感器信号线接口
#define DHTTYPE DHT11 //定义温湿度传感器类型为DHT11
boolean flag=0;
DHT dht(DHTPIN, DHTTYPE);

void setup()
{
    pinMode(BUTTON, INPUT); //定义按钮为输入
    Serial.begin(115200);
    dht.begin();
}

void loop()
{
    float temp = dht.readTemperature();
    Serial.print("当前室温是: ");
    Serial.print(temp);
    Serial.println("°C");

    if (isnan(temp))
    {
        Serial.println("读取DHT11传感器数据失败!");
        delay(1000);
        return;
    }

    if (digitalRead(BUTTON) == 0) //按下停止按钮时, 风扇停止运转
    {
```

DHT11_fan §

```
    if (digitalRead(BUTTON) == 0) //按下停止按钮时, 风扇停止运转
    {
        analogWrite(FANPIN, 0); //将转速值0写入风扇信号口
        delay(1000);
        Serial.println("风扇手动停止运转");
        flag=1;
    }
    else
    {
        if (flag == 0)
        {
            if (temp > 22.5)
            {
                analogWrite(FANPIN, 1500); //将转速值1500写入风扇信号口
                delay(1000);
                Serial.println("风扇正在高速运行");
            }
            else
            {
                if (temp > 21.5 && temp <= 22.5)
                {
                    analogWrite(FANPIN, 500); //将转速值500写入风扇信号口
                    delay(1000);
                    Serial.println("风扇正在低速运行");
                }
            }
            else
            {
                analogWrite(FANPIN, 0); //将转速值0写入风扇信号口
                delay(1000);
                Serial.println("风扇自动停止运转");
            }
        }
        else
        {
            delay(1000);
            Serial.println("风扇手动停止运转");
        }
    }
}
```

接线方式

风扇

- 橙色信号线接开发板信号引脚14
- 红色线接开发板3.3V
- 棕色线接开发板GND

按钮

- OUT引脚接开发板信号引脚0
- VCC引脚接开发板3.3V
- GND引脚接开发板GND

DHT11温湿度传感器

- DAT引脚接开发板信号引脚2
- VCC引脚接开发板5V
- GND引脚接开发板GND

3.3 根据室温控制风扇



COM6

风扇自动停止运转
当前室温是：21.20℃
风扇自动停止运转
当前室温是：21.10℃
风扇自动停止运转
当前室温是：21.10℃
风扇自动停止运转
当前室温是：21.10℃
风扇自动停止运转
当前室温是：21.10℃

实验现象：

室温在21.5℃~22.5℃时，风扇自动启动并低速运转；

室温超过22.5℃时，风扇高速运转；

室温不超过21.5℃时，风扇停止运转。

当风扇运转时，按下停车按钮，风扇停止运转。

实验任务

- o **任务3-1:** 用1个按钮控制舵机启停。按下按钮时舵机连续可逆旋转，松开按钮时舵机停止旋转。串口监视器中显示角度实时值。
- o **任务3-2:** 使用1个旋转电位器控制舵机旋转到任一角度。串口监视器中显示角度实时值。
- o **任务3-3:** 根据土壤湿度传感器测得的湿度值确定舵机是否工作。如果舵机工作，每秒钟在60°的扇形范围内连续可逆旋转。可以在串口监视器中查看土壤湿度实时值。
- o **任务3-4:** 用1个按钮控制风扇启停。串口监视器中显示风扇状态。
- o **任务3-5:** 使用1个旋转电位器控制风扇转速。串口监视器中显示转速值。
- o **任务3-6:** 根据温湿度传感器测得的温度值确定风扇是否工作。当温度超过26℃时，风扇自动启动；如果温度不超过30℃，风扇低速运转；如果温度超过30℃，风扇高速运转；如果温度不超过26℃，风扇自动停止运转。用1个按钮作为停止按钮，当风扇运转时，按下停止按钮，风扇停止运转。
- o **任务3-7:** 智能风扇。
 - o (1) 共需要4个按钮。1个按钮负责风扇启动；1个按钮负责风扇停机；1个按钮负责风扇高低速切换；1个按钮负责风扇是否摇头。
 - o (2) 风扇可以自动运行，也可以手动运行。手动运行优先级高于自动运行。
 - o (3) 手动运行：随时可以通过按下启动按钮，启动风扇；随时可以通过按下停止按钮，使风扇停止运转。使用风扇高低速切换按钮，可以使风扇高速运转或低速运转。使用风扇摇头按钮，可以使舵机连续可逆运行（模拟摇头）或不运行。
 - o (4) 自动运行：根据温湿度传感器测得的温度值确定风扇是否工作。当温度超过26℃时，风扇自动启动；如果温度不超过30℃，风扇低速运转；如果温度超过30℃，风扇高速运转；如果温度不超过26℃，风扇自动停止运转。如果风扇正在自动运行，当按下停止按钮时，风扇停止运转。自动运行时默认风扇摇头（舵机连续可逆运行）。
 - o (5) 串口监视器中显示智能风扇的运行状态，如风扇运行还是停止、自动运行还是手动运行、高速运行还是低速运行、是否在摇头、当前室温等。
 - o (6) 程序中需带有小组成员个人身份标识，如姓名首字母缩写、姓氏拼音、名字拼音、学号后3位等。